

A Negative Result for Training-Free Temporal Visual Caching in OpenVLA-OFT

Ved Dwivedi, Cheng Yang, and Bo Yuan

Abstract—Vision–language–action (VLA) policies repeatedly process similar visual observations during robot manipulation, making temporal reuse a potential approach for reducing inference cost. However, reused representations may omit task-relevant changes and alter subsequent closed-loop behavior. We present a systematic evaluation of three training-free temporal reuse strategies in OpenVLA-OFT on LIBERO: complete visual-prefix reuse, scene-camera reuse with wrist-camera refresh, and selective reuse of visual key–value states across transformer layers and image regions. Complete-prefix reuse was evaluated over 1,160 closed-loop episodes. Settings that skipped visual refresh on 34.69–83.68% of policy queries achieved only 0–52% task success. Conservative refresh rules achieved 69 successes in 70 episodes but skipped visual refresh on only 0.95% of policy queries. Scene-camera reuse matched the observed success count of batched dense inference at 67/70 while reusing the scene representation on 25.24% of queries, but it provided no end-to-end latency improvement. Mean query time was 1,190.97 ms, compared with 1,188.28 ms for dense inference. Fine-grained key–value reuse reduced median complete-query time by 22.60% in a controlled systems test but achieved 0/120 closed-loop successes, compared with 68/120 for dense inference from the same initial conditions. These findings show that training-free reuse at the tested representation boundaries did not provide a reliable success–latency tradeoff. They motivate reuse mechanisms that preserve current task-relevant information and policies trained explicitly to tolerate stale representations.

Index Terms—Vision–language–action models, robot learning, inference efficiency, temporal caching, closed-loop control.

I. INTRODUCTION

VISION–LANGUAGE–ACTION (VLA) models map visual observations, language instructions, and robot state to actions. Recent systems have improved generalization across manipulation tasks, but their large visual encoders and language-model backbones make inference expensive [1]–[3]. OpenVLA-OFT improves action-generation efficiency through continuous action prediction, parallel decoding, and action chunking [4]. Even with these improvements, the model processes the visual input again at every policy query.

Consecutive observations in manipulation often share the same background and many of the same objects. This temporal redundancy motivates caching visual tokens or transformer key–value (K/V) states across queries. VLA-Cache, for example, reports that static visual tokens can be reused while task-relevant tokens are recomputed [5]. Other work accelerates VLA inference through visual token selection, model pruning, or reuse within an iterative action head [6]. These results

establish that substantial redundancy exists. They also raise a system-specific question: when visual features are reused in an autoregressive VLA with action chunking, does the resulting policy preserve closed-loop task performance?

The difficulty is not captured by image similarity alone. A small change in a cached representation can change an action chunk. That action changes the robot state and the next camera observation. Later cache decisions are then made on a trajectory generated by the modified policy rather than the dense policy. Offline comparisons on dense-policy trajectories cannot reproduce this feedback.

We study this issue with one released OpenVLA-OFT checkpoint and the LIBERO simulator [7]. The experiments progress from coarse to fine reuse. We first cache the complete projected representation of both cameras and trigger refreshes using image, robot-state, action, and cache-age signals. We then keep the wrist-camera representation current and reuse only the scene-camera representation. Finally, we reuse selected K/V states by layer and image region while recording the physical query from which every reused state originated.

The experiments consistently expose a reliability–efficiency boundary. Aggressive complete-prefix reuse reduces task success. Conservative refresh rules preserve success only when reuse becomes rare. Camera-selective reuse preserves success at a useful logical reuse rate, but its latency advantage disappears against a batched dense implementation. Fine-grained K/V reuse yields measurable computational savings but fails closed-loop control in the final paired evaluation. Together, these experiments provide an empirical characterization of a failure mode that is easy to miss when caching is evaluated only through token counts, component timings, or offline action similarity.

II. RELATED WORK

A. Vision–language–action policies

RT-1 and RT-2 demonstrated large-scale language-conditioned visuomotor policies and transfer from vision–language pretraining [1], [2]. OpenVLA provides an open 7B-parameter VLA built from fused DINOv2 and SigLIP visual features and a Llama-family language backbone [3]. OpenVLA-OFT replaces discrete action-token generation with a continuous action head and predicts action chunks in parallel [4]. We use the released OpenVLA-OFT inference stack because it provides a public checkpoint, an official evaluator, and a strong dense baseline on LIBERO.

Ved Dwivedi is with John P. Stevens High School, Edison, NJ, USA.

Cheng Yang and Bo Yuan are with the Department of Electrical and Computer Engineering, Rutgers University–New Brunswick, Piscataway, NJ, USA.

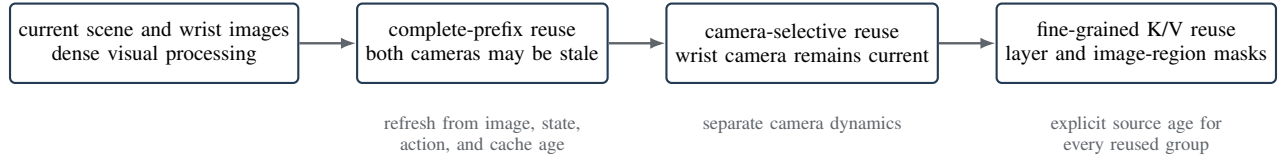


Fig. 1. The three temporal reuse mechanisms evaluated in this study. Each mechanism retains more current information than the one before it.

B. Efficient VLA inference

VLA inference can be accelerated by reducing model depth, processing fewer current-frame tokens, or reusing intermediate computation. EfficientVLA combines language-layer pruning, task-aware visual token selection, and feature caching within a diffusion action head [6]. VLA-Cache identifies visually stable tokens, excludes task-relevant regions using attention, and varies reuse across decoder layers [5]. Recent work also studies learned token-selection policies that are optimized with the task objective [8]. These methods differ in architecture, cache location, and evaluation platform. Their published speedups are therefore not direct numerical baselines for our OpenVLA-OFT implementation.

Our study focuses on cross-query reuse in a closed-loop, action-chunked policy. The main distinction is between current-frame compression, which still derives every retained token from the current observation, and temporal reuse, which inserts representations computed from an earlier observation. The latter introduces representation age and can alter the state distribution encountered during control.

C. Closed-loop distribution shift

Sequential imitation learning distinguishes the state distribution generated by the reference policy from the distribution generated by a modified policy [9]. Let d_{π_D} denote the state distribution induced by dense inference and d_{π_C} the distribution induced by a cached policy. An offline comparison generally samples from d_{π_D} , whereas deployment evaluates states from d_{π_C} . Once cached inference changes an action,

$$d_{\pi_C}(s) \neq d_{\pi_D}(s) \quad (1)$$

in general. This motivates direct closed-loop evaluation rather than treating offline action similarity as the final reliability measure.

III. PROBLEM SETTING

A. Dense policy

Let q index policy queries. At query q , OpenVLA-OFT receives a language instruction x , a scene image o_q^s , a wrist image o_q^w , and proprioceptive state p_q . The visual encoder and projector produce visual tokens

$$V_q = \mathcal{V}_\theta(o_q^s, o_q^w). \quad (2)$$

The transformer and continuous action head produce an eight-step, seven-dimensional action chunk

$$A_q^D = h_\theta(\mathcal{T}_\theta(x, V_q, p_q)) \in \mathbb{R}^{8 \times 7}. \quad (3)$$

The evaluator normally executes all eight actions before requesting another chunk. A changed prediction can therefore affect several environment transitions before the next cache decision.

B. Temporal K/V reuse

At transformer layer ℓ and visual group g , let $K_{q,\ell,g}$ and $U_{q,\ell,g}$ denote the K/V states computed from the current query. A binary mask $M_{q,\ell,g} = 1$ selects the current states. If $M_{q,\ell,g} = 0$, the cache uses states from a physical source query $\sigma(q, \ell, g) < q$:

$$\tilde{K}_{q,\ell,g} = M_{q,\ell,g} K_{q,\ell,g} + (1 - M_{q,\ell,g}) K_{\sigma(q,\ell,g),\ell,g}, \quad (4)$$

$$\tilde{U}_{q,\ell,g} = M_{q,\ell,g} U_{q,\ell,g} + (1 - M_{q,\ell,g}) U_{\sigma(q,\ell,g),\ell,g}. \quad (5)$$

The source age is $a_{q,\ell,g} = q - \sigma(q, \ell, g)$. Recursive reuse can create one transformer state whose visual groups originate from different physical times.

C. Evaluation measures

For a fixed set of N episodes, terminal task success is

$$\hat{S}(m) = \frac{1}{N} \sum_{i=1}^N \mathbb{1}\{\text{episode } i \text{ succeeds under method } m\}. \quad (6)$$

We report exact two-sided 95% binomial intervals for the central success results [10]. These intervals do not model clustering by task or initial state.

For timing experiments, end-to-end improvement relative to dense inference is

$$G_T(m) = 1 - \frac{T_m}{T_D}, \quad (7)$$

where T_m includes controller logic, cache management, synchronization, and data movement. We separately report the fraction of policy queries that reuse visual computation. Logical reuse and component-level CUDA time are mechanism measurements, not substitutes for end-to-end latency.

IV. TEMPORAL REUSE METHODS

A. Complete visual-prefix reuse

The first method stores the projected scene and wrist visual tokens from the most recent refresh query τ_q . It measures image change d_q^{img} , normalized proprioceptive change d_q^{state} , recent action change d_q^{act} , and cache age $q - \tau_q$. The visual representation is refreshed when

$$r_q = \mathbb{1}\left[d_q^{\text{img}} > \gamma_I \vee d_q^{\text{state}} > \gamma_S \vee d_q^{\text{act}} > \gamma_A \vee q - \tau_q \geq H\right]. \quad (8)$$

If $r_q = 0$, the policy receives V_{τ_q} in place of V_q . We first use the average image change across both cameras. We then evaluate more conservative variants that threshold each camera separately, group state and action coordinates by physical meaning, prevent consecutive reuse, and force refreshes near gripper or motion-direction transitions.

B. Camera-selective reuse

Diagnostics from the complete-prefix experiments show that the wrist image often changes while the scene image remains stable. We therefore keep the wrist representation current and permit reuse only for the scene representation:

$$\tilde{V}_q = [V_{\tau_q}^s; V_q^w]. \quad (9)$$

On refresh queries, both cameras are processed as one batch. This batched dense path is the timing baseline because it avoids duplicated framework overhead without introducing stale information. The cache controller permits at most one reuse query before the next mandatory refresh.

C. Fine-grained K/V reuse

The final cache partitions each camera into a 4×4 grid and applies observation-dependent reuse masks at transformer layers 2, 6, 9, and 11. The maximum number of reusable scene-camera tokens increases from 48 to 192 of 256 tokens across these layers. The corresponding wrist-camera limits increase from 32 to 128 tokens. At least one region per camera remains current, scene states are at most four queries old, wrist states are at most two queries old, and the cache periodically resets to dense computation. The implementation records $\sigma(q, \ell, g)$ for every reused group so that the physical age in (5) remains explicit.

We also explored whether an offline model could predict when fine-grained reuse would increase action error. Those exploratory targets were later found to depend on a custom action-extraction path that did not match the released evaluator. We therefore exclude the associated action-quality and risk-prediction results from the scientific analysis. Before the final fine-grained experiment, the custom dense path was corrected and independently verified to reproduce the released evaluator's hidden states and actions exactly.

V. EXPERIMENTAL PROTOCOL

A. Model, benchmark, and hardware

All experiments use the released `openvla-7b-ofit-libero-four-suite` checkpoint, two RGB cameras, current proprioception, and the official LIBERO terminal success function. The model runs on one NVIDIA TITAN RTX. The full-refresh experiments use LIBERO-Spatial. The paired camera-selective experiment uses LIBERO-Object. The final fine-grained cache experiment uses 30 initial conditions from each of the Spatial, Object, Goal, and LIBERO-10 suites.

The method families were developed sequentially and use different evaluation populations. We report their sample sizes separately and do not pool them into a single estimator. Paired comparisons use the same initial condition for dense and cached inference.

B. Timing baseline

A separate 50-episode pilot measures 662 steady policy queries. Full refresh succeeds in 49/50 episodes. The visual encoder and projector require 201.19 ms per query, equal to 15.874% of measured query CUDA time. Even a zero-overhead method that removed all visual work would therefore have a maximum speedup of approximately $1.189\times$ in this implementation. The pilot is not included in the 1,160 complete-prefix evaluation episodes.

C. Implementation validation

The final K/V experiment includes an independent comparison with the released OpenVLA-OFT evaluator. The test compares the hidden-state slice used by the continuous action head, normalized actions, and unnormalized actions. All three agree with maximum absolute error zero. Incomplete executions and measurements generated before this check are excluded from the reported final cache result.

VI. RESULTS

A. Complete-prefix reuse

Dense inference succeeds in all 100 baseline episodes used for the initial complete-prefix comparison. Across nine reuse settings, visual refresh is skipped on 34.69–83.68% of policy queries, while terminal success ranges from 52/100 to 0/100. The best of these settings is 48 percentage points below dense inference.

Conservative refresh rules improve reliability but sharply reduce reuse. A configuration with no reuse succeeds in 30/30 episodes. Configurations with 6.72% and 10.57% reuse succeed in 29/30 and 27/30 episodes. The most conservative final configuration succeeds in 69/70 episodes but reuses the visual prefix on only 9 of 944 policy queries, or 0.9534%.

The first reused query changes the predicted action in all 87 comparable episodes for which this diagnostic is available. Offline replay on dense-policy trajectories underestimates online reuse for every permissive setting. Camera averaging also obscures asymmetric change: 374 of 783 reuse decisions contain a threshold exceedance in at least one camera, and 372 of these cases are caused only by the wrist camera. These observations motivate the camera-selective experiment.

B. Camera-selective reuse

The initial camera-selective screen evaluates three 30-episode settings. They reuse the scene representation on 26.06%, 47.40%, and 49.44% of queries and obtain 29/30, 24/30, and 23/30 successes, respectively. The most conservative setting therefore advances to a paired 70-episode comparison.

In the paired experiment, camera-selective reuse and batched dense inference each succeed in 67/70 episodes. The reuse policy retains an earlier scene representation on 25.24% of queries. Relative to a sequential dense implementation, its mean wall time is 3.96% lower. The relevant baseline is the batched dense implementation, however, because both camera

TABLE I
EVALUATION POPULATIONS AND PRIMARY MEASUREMENTS.

Reuse mechanism	Benchmark population	Closed-loop sample	Primary reliability measure	Efficiency measure
Complete visual prefix	LIBERO-Spatial development states	1,160 primary episodes	Terminal task success across threshold settings	Fraction of queries that skip visual refresh
Camera-selective visual prefix	LIBERO-Object paired states	70 dense + 70 cached episodes	Paired terminal success	Complete wall time and visual CUDA time
Fine-grained K/V states	30 paired states from each of four LIBERO suites	120 dense + 120 cached episodes	Paired terminal success by suite	Median complete-query time reduction in the systems test

TABLE II
REPRESENTATIVE COMPLETE-PREFIX RESULTS. INTERVALS ARE EXACT TWO-SIDED 95% BINOMIAL INTERVALS.

Configuration	Success	95% interval	Reuse
Dense reference	100/100	96.38–100%	0%
Best permissive setting	52/100	41.78–62.10%	34.69%
Highest-reuse setting	0/100	0–3.62%	83.68%
Conservative, no reuse	30/30	88.43–100%	0%
Conservative, moderate	29/30	82.78–99.92%	6.72%
Conservative, higher	27/30	73.47–97.89%	10.57%
Final conservative setting	69/70	92.30–99.96%	0.95%

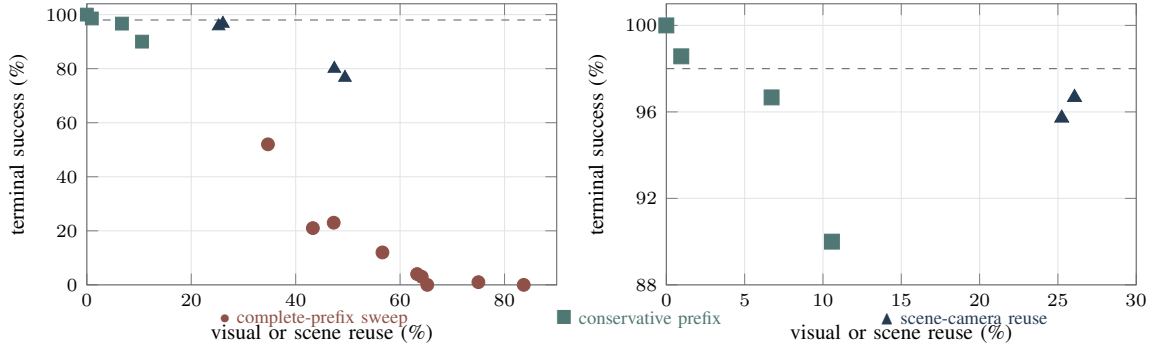


Fig. 2. Observed success–reuse operating points. The right panel enlarges the high-success region. Points summarize separate experimental populations and are therefore descriptive rather than randomized head-to-head estimates.

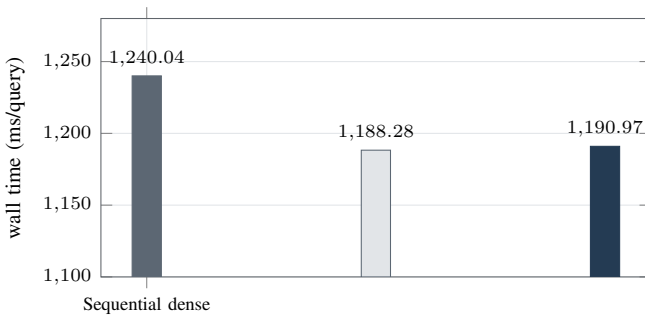


Fig. 3. Mean complete-query wall time in the paired camera-selective experiment. Reusing the scene representation does not improve latency relative to batched dense inference.

images can be processed together without using stale information. Against this baseline, camera-selective reuse reduces visual CUDA time by 8.46% but increases mean complete-query wall time from 1188.28 to 1190.97 ms. The 2.69 ms difference corresponds to a 0.23% slowdown.

C. Fine-grained K/V reuse

The validated fine-grained cache reduces median complete-query time by 22.5969% in a 97-call systems test. The paired closed-loop evaluation then runs dense and cached inference from the same 120 initial conditions, producing 240 episodes and 8,525 model queries. Dense inference succeeds in 68/120 episodes. Cached inference succeeds in 0/120 episodes, including 0/30 in each LIBERO suite. The overall dense–cache difference is 56.67 percentage points.

VII. DISCUSSION

A. Visual similarity did not imply policy equivalence

The complete-prefix results show that visually similar observations can still require different actions. All 87 examined episodes change their predicted action at the first reused query. The wrist-camera diagnostic gives a concrete explanation for part of this sensitivity. The wrist view changes with end-effector motion and object contact even when the global scene remains stable. Averaging the two camera-change signals hides this local change.

TABLE III
 PAIRED CLOSED-LOOP RESULTS FOR FINE-GRAINED K/V REUSE. INTERVALS ARE EXACT TWO-SIDED 95% BINOMIAL INTERVALS.

Suite	Dense (95% interval)	Cache (95% interval)	Difference
Spatial	14/30 (28.34–65.67%)	0/30 (0–11.57%)	–46.67 percentage points
Object	27/30 (73.47–97.89%)	0/30 (0–11.57%)	–90.00 percentage points
Goal	13/30 (25.46–62.57%)	0/30 (0–11.57%)	–43.33 percentage points
LIBERO-10	14/30 (28.34–65.67%)	0/30 (0–11.57%)	–46.67 percentage points
Overall	68/120 (47.31–65.68%)	0/120 (0–3.03%)	–56.67 percentage points

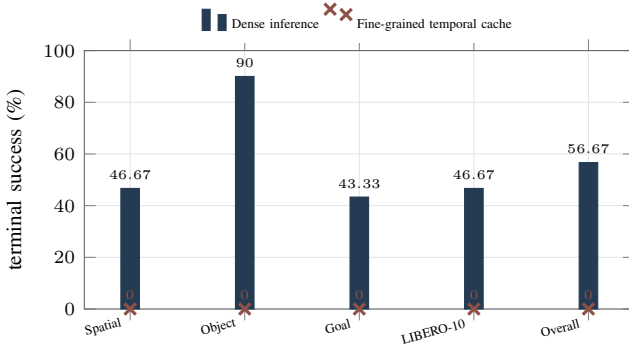


Fig. 4. Paired closed-loop success for dense inference and the validated fine-grained temporal cache. Each suite contains 30 episodes per condition.

Keeping the wrist representation current substantially improves reliability. The paired camera-selective policy matches dense success while reusing the scene representation on one quarter of policy queries. This result shows that temporal reuse is not uniformly harmful. It also shows why success and speed must be measured together. The extra cache-control and execution paths cost more than the visual work they remove when compared with a batched dense baseline.

B. Recursive K/V reuse changes the control state

The fine-grained cache does not simply store independent image descriptors. At every transformer layer, the visual states interact with the instruction, proprioception, and action-query states. Reusing groups with different source ages therefore constructs an internal state that did not occur in dense inference. If that state changes an action chunk, the robot reaches a different observation. Recursive reuse then carries information from this altered trajectory into later queries.

The final result is therefore stronger than an offline action-distance observation. The cache reduces measured query time yet fails every closed-loop episode. This does not prove that all K/V caching strategies will fail. It shows that the tested observation-dependent, mixed-age cache is not a usable operating point for this checkpoint and evaluator.

C. The dense baseline determines whether a speedup is real

The camera-selective experiment initially appears faster than sequential full refresh. Batching the two current camera images removes much of that apparent improvement without introducing staleness. A temporal cache should therefore be compared with a dense path that already uses batching, compiler optimization, quantization, and other compatible current-frame improvements. Otherwise, the reported speedup can

reflect an avoidable implementation overhead rather than the caching method.

D. Implications for future methods

The results favor two directions. The first is current-frame compression, including task-aware token selection, token merging, quantization, and adaptive spatial resolution. These methods reduce computation while keeping the retained information tied to the present observation.

The second direction is cache-aware learning. If temporal reuse is required, the model can be trained with the same stale and recursively updated states that it will encounter at deployment. A learned selector or internal adapter could use explicit source age and task-relevance signals. Closed-loop evaluation should be introduced early because an offline reduction in action error does not guarantee recovery of terminal task success.

We considered a small residual adapter that would correct cached actions using current visual summaries and source-age information. It was not trained. The validated fine-grained cache achieved 0/120 closed-loop successes, so a low-dimensional correction would have been asked to recover a policy with no demonstrated control capability. The present experiments therefore do not evaluate learned action correction as a method.

VIII. LIMITATIONS

The experiments use one OpenVLA-OFT checkpoint, one simulator benchmark family, and one GPU class. The results may not transfer to other VLA architectures, action heads, camera layouts, or real robots. The method families were developed sequentially and use different populations, so the success–reuse frontier is descriptive rather than a randomized comparison.

Dense inference in the final four-suite experiment succeeds in 56.67% of episodes, below the published OpenVLA-OFT four-suite average [4]. This difference limits conclusions about absolute checkpoint performance in that experiment. It does not remove the paired observation that the temporal cache succeeds in 0 of the same 120 initial conditions.

The binomial intervals treat episodes as exchangeable trials and do not model dependence among tasks or initial states. We do not measure collisions, constraint violations, or formal robot safety. In this paper, reliability refers only to terminal task success and explicitly stated action-equivalence checks.

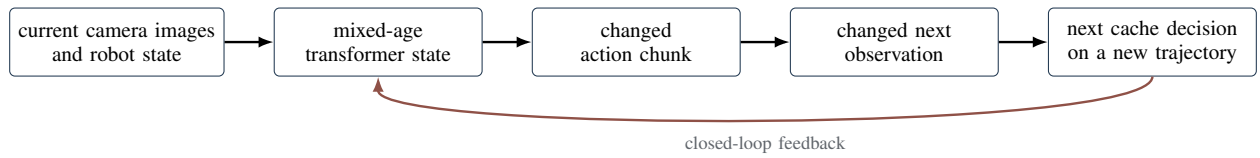


Fig. 5. Temporal reuse can alter the trajectory on which later reuse decisions are made. Offline evaluation on dense-policy trajectories omits this feedback.

IX. REPRODUCIBILITY AND DATA AVAILABILITY

The public repository at <https://github.com/vwdr/SAVR> contains the cache implementations, evaluation configurations, analysis scripts, machine-readable summaries, and document sources. The repository records the exact experimental identifiers and artifact hashes. Large rollout artifacts are not yet deposited in a public archival repository. The final reported K/V result includes only runs completed after independent agreement with the released OpenVLA-OFT evaluator was established.

X. CONCLUSION

This study evaluates training-free temporal visual caching in an action-chunked VLA under closed-loop control. Complete-prefix reuse reduces success at useful reuse rates. Conservative refresh rules recover success only by nearly eliminating reuse. Reusing only the scene-camera representation preserves success but does not reduce end-to-end latency relative to batched dense inference. Fine-grained K/V reuse reduces measured query time but fails all 120 paired closed-loop episodes.

The evidence does not support temporal caching at the tested representation boundaries in OpenVLA-OFT. It identifies the central requirement for a stronger method: computational savings must be obtained while preserving information about the current control state. Current-frame compression and cache-aware training are more plausible paths than additional threshold tuning on representations computed from earlier observations.

REFERENCES

- [1] A. Brohan *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Proc. Robotics: Science and Systems*, 2023.
- [2] B. Zitkovich *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proc. Conf. Robot Learning*, ser. Proc. Mach. Learn. Res., vol. 229, 2023, pp. 2165–2183.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” in *Proc. Conf. Robot Learning*, ser. Proc. Mach. Learn. Res., vol. 270, 2025, pp. 2679–2713.
- [4] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Proc. Robotics: Science and Systems*, 2025.
- [5] S. Xu, Y. Wang, C. Xia, D. Zhu, T. Huang, and C. Xu, “VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 164 448–164 473.
- [6] Y. Yang, Y. Wang, Z. Wen, L. Zhongwei, C. Zou, Z. Zhang, C. Wen, and L. Zhang, “EfficientVLA: Training-free acceleration and compression for vision-language-action models,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 40 891–40 914.
- [7] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 44 776–44 791.
- [8] Y. Wei, J. Fan, J. Guo, R. Zhen, R. Shao, X. Su, Z. Xie, H. Yao, and S. Yang, “Learning to accelerate vision-language-action models through adaptive visual token caching,” *arXiv preprint arXiv:2602.00686*, 2026.
- [9] S. Ross, G. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. Int. Conf. Artificial Intelligence and Statistics*, ser. Proc. Mach. Learn. Res., vol. 15, 2011, pp. 627–635.
- [10] C. J. Clopper and E. S. Pearson, “The use of confidence or fiducial limits illustrated in the case of the binomial,” *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.